



사케 검색 봇 - 왕초보 설치 가이드

이 가이드는 서버, 네트워크, 데이터베이스를 전혀 모르시는 분들을 위해 작성되었습니다. 순서대로 천천히 따라 하시면 누구나 설치할 수 있습니다.



1단계: 필수 프로그램 설치

이 프로젝트를 실행하려면 내 컴퓨터에 기본 도구들이 설치되어 있어야 합니다.

1. VS Code 설치 (코드 편집기)

- [VS Code 다운로드](#)에 접속해서 다운로드 및 설치하세요.
- 설치 후 실행하여 `sake-import-checker` 폴더를 엽니다. (File -> Open Folder)

2. Node.js 설치 (자바스크립트 실행기)

- [Node.js 다운로드](#)에 접속합니다.
- **LTS 버전** (안정화 버전)을 다운로드하여 설치합니다.
- 설치가 잘 되었는지 확인하려면:
 - VS Code에서 터미널을 엽니다 (`Ctrl + ~` 키 또는 메뉴의 Terminal -> New Terminal).
 - `node -v` 라고 입력하고 엔터를 칩니다. `v18.xx.x` 같은 숫자가 나오면 성공입니다.

3. Git 설치 (버전 관리 도구)

- [Git 다운로드](#)에서 설치합니다.
- 설치 옵션은 모두 "Next"만 눌러도 됩니다.



2단계: 서비스 계정 만들기

무료로 서버와 데이터베이스를 사용하기 위해 4곳의 회원가입이 필요합니다.

1. Cloudflare (서버)

- [Cloudflare 가입](#)
- 이메일 인증까지 완료하세요.

2. Supabase (데이터베이스)

- [Supabase 가입](#)
- GitHub 아이디로 로그인하면 편합니다.
- 새 프로젝트 만들기:
 - [New Project] 클릭 -> Organization 선택 (없으면 생성)
 - **Name:** sake-bot (아무거나 상관없음)
 - **Database Password:** [중요] 꼭 메모장에 적어두세요! (나중에 못 봅니다)
 - **Region:** Northeast Asia (Seoul) 선택
 - [Create new project] 클릭하고 약 2~3분 기다리면 초록색 "Active" 상태가 됩니다.

3. Google AI Studio (AI 눈)

- [Google AI Studio](#) 접속
- 구글 아이디로 로그인
- [Get API Key] -> [Create API key] 클릭
- 생성된 키(AIza... 로 시작하는 긴 문자열)를 복사해서 메모장에 저장하세요.

4. Telegram (메신저)

- 텔레그램 앱을 켵니다.
- 검색창에 BotFather 검색 (파란 체크 표시 있는 공식 봇)
- 채팅방 들어가서 /start 입력
- /newbot 입력
- 봇 이름 입력 (예: 우리집 사케봇)
- 봇 아이디 입력 (예: my_sake_bot_123 -> 반드시 bot 으로 끝나야 함)
- 성공하면 **HTTP API Token**이라는 긴 문자열을 줍니다. 복사해서 메모장에 저장하세요.



3단계: 데이터베이스 설정하기 (최신 HNSW 인덱스 적용)

Supabase 대시보드에서 설정을 마쳐야 합니다. 이 설정은 검색 정확도를 높여줍니다.

1. 왼쪽 메뉴에서 **SQL Editor** 아이콘(종이 2장 겹친 모양) 클릭
2. 빈 페이지에 아래 내용을 그대로 복사해서 붙여넣습니다.

-- 여기부터 복사하세요

```
CREATE EXTENSION IF NOT EXISTS vector;
```

```
CREATE TABLE IF NOT EXISTS sake_imports (  
  id BIGSERIAL PRIMARY KEY,  
  reported_product_name TEXT NOT NULL,  
  category TEXT,  
  exporter TEXT,  
  origin_country TEXT,  
  raw_importer_name TEXT,  
  value NUMERIC,  
  volume NUMERIC,  
  unit_price NUMERIC,  
  name_embedding vector(768),  
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),  
  updated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()  
);
```

-- 검색 정확도를 위한 메모리 설정

```
SET maintenance_work_mem = '256MB';
```

-- HNSW 인덱스 생성 (정확도 99%)

```
CREATE INDEX IF NOT EXISTS idx_sake_imports_embedding  
  ON sake_imports  
  USING hnsw (name_embedding vector_cosine_ops)  
  WITH (m = 24, ef_construction = 128);
```

```
CREATE TABLE IF NOT EXISTS upload_progress (  
  session_id TEXT PRIMARY KEY,  
  current_count INT DEFAULT 0,  
  total_count INT DEFAULT 0,  
  status TEXT DEFAULT 'pending',  
  error_message TEXT,  
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),  
  updated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()  
);
```

```
CREATE TABLE IF NOT EXISTS search_logs (  
  id BIGSERIAL PRIMARY KEY,  
  user_id BIGINT,  
  chat_id BIGINT,  
  query_text TEXT,  
  photo_file_id TEXT,
```

```
extracted_info JSONB,  
matched_product_id BIGINT,  
confidence_score FLOAT,  
response_time_ms INT,  
created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()  
);
```

```
CREATE OR REPLACE FUNCTION search_products(  
  query_embedding vector(768),  
  match_count INT DEFAULT 10,  
  similarity_threshold FLOAT DEFAULT 0.5  
)
```

```
RETURNS TABLE (  
  id BIGINT,  
  reported_product_name TEXT,  
  category TEXT,  
  exporter TEXT,  
  origin_country TEXT,  
  raw_importer_name TEXT,  
  value NUMERIC,  
  volume NUMERIC,  
  unit_price NUMERIC,  
  similarity FLOAT  
)
```

```
LANGUAGE plpgsql
```

```
AS $$
```

```
BEGIN
```

```
  -- HNSW 검색 최적화
```

```
  SET LOCAL hnsw.ef_search = 100;
```

```
RETURN QUERY
```

```
SELECT
```

```
  si.id,  
  si.reported_product_name,  
  si.category,  
  si.exporter,  
  si.origin_country,  
  si.raw_importer_name,  
  si.value,  
  si.volume,  
  si.unit_price,
```

```
  1 - (si.name_embedding <=> query_embedding) AS similarity
```

```
FROM sake_imports si
```

```

WHERE 1 - (si.name_embedding <=> query_embedding) > similarity_threshold
ORDER BY si.name_embedding <=> query_embedding
LIMIT match_count;
END;
$$;

```

```

CREATE OR REPLACE FUNCTION get_stats()
RETURNS TABLE (
    total_products BIGINT,
    last_updated TIMESTAMP WITH TIME ZONE,
    top_exporters JSONB
)
LANGUAGE plpgsql
AS $$
BEGIN
    RETURN QUERY
    SELECT
        COUNT(*)::BIGINT AS total_products,
        MAX(updated_at) AS last_updated,
        (
            SELECT jsonb_agg(row_to_json(t))
            FROM (
                SELECT exporter, COUNT(*) AS count
                FROM sake_imports
                GROUP BY exporter
                ORDER BY count DESC
                LIMIT 5
            ) t
        ) AS top_exporters
    FROM sake_imports;
END;
$$;

```

```

CREATE OR REPLACE FUNCTION update_updated_at()
RETURNS TRIGGER AS $$
BEGIN
    NEW.updated_at = NOW();
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

```

```

CREATE TRIGGER trigger_sake_imports_updated
BEFORE UPDATE ON sake_imports

```

```

FOR EACH ROW
EXECUTE FUNCTION update_updated_at();

CREATE TRIGGER trigger_upload_progress_updated
BEFORE UPDATE ON upload_progress
FOR EACH ROW
EXECUTE FUNCTION update_updated_at();
-- 여기까지 복사하세요

```

3. 우측 하단 **Run** 버튼 클릭 -> "Success" 메시지가 나오면 성공!

4. 이제 연결 정보를 가져옵니다.

- 왼쪽 메뉴 **Project Settings** (톱니바퀴) -> **API** 클릭
- **Project URL**: 복사해서 메모장에 저장 (SUPABASE_URL)
- **Project API keys** -> **anon public**: 복사해서 메모장에 저장 (SUPABASE_KEY)

4단계: 진짜 배포하기 (한방 스크립트)

이제 복잡한 명령어 대신 제가 만들어둔 스크립트를 실행하면 됩니다.

1. 터미널 준비

VS Code 터미널에서 `sake-import-checker` 폴더에 있는지 확인하세요.

(경로가 `.../sake-import-checker` 로 끝나야 합니다)

2. 배포 스크립트 실행

터미널에 다음 명령어를 입력하고 엔터를 칩니다.

```
./scripts/deploy.sh
```

3. 질문에 답하기

스크립트가 실행되면 몇 가지 질문을 합니다. 메모장을 보며 입력하세요.

1. **로그인 요청**: 브라우저가 열리면서 Cloudflare 로그인을 하라고 할 수 있습니다. "Allow"를 눌러주세요.
2. **Secrets 확인**:
 - `TELEGRAM_BOT_TOKEN` : 텔레그램 BotFather가 준 토큰 붙여넣기

- GEMINI_API_KEY : 구글에서 받은 AI 키 붙여넣기
- SUPABASE_URL : Supabase URL 붙여넣기
- SUPABASE_KEY : Supabase anon key 붙여넣기
- ADMIN_PASSWORD : 관리자 페이지 접속용 비밀번호 (원하는 거 아무거나 설정)

3. 자동 배포: 위 정보를 다 입력하면 알아서 서버를 설치하고 배포합니다. (약 1~2분 소요)

4. 웹훅 연결:

- 마지막으로 "Worker URL을 입력하세요" 라고 나오면, 위에서 배포 완료 시 출력된 URL (예: `https://sake-import-checker.xxx.workers.dev`)을 복사해서 붙여넣습니다.



5단계: 완료 확인

1. 관리자 페이지 접속

- 위에서 나온 URL 뒤에 `/admin` 은 아니고 별도의 Admin 페이지 URL이 나왔을 겁니다.
- 보통 `https://sake-admin.pages.dev` 형태입니다.
- 접속해서 설정한 비밀번호로 로그인해봅니다.
- 엑셀 파일을 업로드해봅니다. (보안 강화로 인해 로그인이 필수입니다)

2. 텔레그램 봇 테스트

- 텔레그램에서 내 봇을 검색해서 채팅을 겁니다.
- `/start` 라고 쳐봅니다. "사케 검색 봇입니다" 라고 답장이 오면 성공!
- 사케 라벨 사진을 보내봅니다.



문제 해결

Q: 터미널에서 에러가 빨간색으로 너무 많이 나와요!

A: `npm install` 명령어를 backend 폴더 안에서 실행했는지 확인하세요.

```
cd backend
npm install
cd ..
```


Q: 봇이 아무 반응이 없어요.

A: Webhook 설정이 꼬였을 수 있습니다. 스크립트를 다시 실행해서 마지막 단계(4번)만 다시 해보거나, 수동으로 웹훅을 설정해보세요.

Q: 엑셀 업로드했는데 진행률이 안 올라가요.

A: Cloudflare와 Supabase 연결 문제일 수 있습니다. backend 폴더에서 wrangler tail 명령어를 입력하고, 엑셀을 다시 업로드해보면 에러 로그를 볼 수 있습니다.

Q: 검색 결과가 이상해요.

A: 데이터베이스 설정(3단계)을 다시 한 번 실행해보세요. 특히 HNSW 인덱스가 잘 생성되었는지 확인이 필요합니다.